

Exam Presentation - ATCG2

Henning Lehmann

February 2026

Outline

1 Part 1: Mesh Synthesis from Point Clouds

2 Part 2: The Laplacian Operator

Part 1: Mesh Synthesis from Point Clouds

Problem Statement

- Have: a point cloud $\mathcal{P} = \{p_1, \dots, p_n\}$, $p_i \in \mathbb{R}^3$, sampled from a surface F

Problem Statement

- Have: a point cloud $\mathcal{P} = \{p_1, \dots, p_n\}$, $p_i \in \mathbb{R}^3$, sampled from a surface F
- Want: an explicit representation (a triangle mesh) of F

Problem Statement

- Have: a point cloud $\mathcal{P} = \{p_1, \dots, p_n\}$, $p_i \in \mathbb{R}^3$, sampled from a surface F
- Want: an explicit representation (a triangle mesh) of F
- Idea: estimate a signed distance field from the point cloud, then use marching cubes (or something similar) to approximate F .

Fast Marching

Signed distance at a grid point:

$$d(g, \mathcal{P}) = (g - c)^T n(c)$$

Normal Estimation

Neighborhood of sample point p :

$$\mathcal{N}(p) = \{p_i \mid \|p_i - p\|_2 < \delta\}.$$

Least squares fit for (hyper)plane:

$$\arg \min_{n, d} \frac{1}{|\mathcal{N}|} \sum_{p \in \mathcal{N}} \|n^T p - d\|^2.$$

Minimizing Covariance

$$\arg \min_v \sigma^2(v) = \frac{1}{|\mathcal{N}|} \sum_{p \in \mathcal{N}} (v^T (p - \bar{p}))^2$$

Minimizing Covariance

$$\begin{aligned}\arg \min_v \sigma^2(v) &= \frac{1}{|\mathcal{N}|} \sum_{p \in \mathcal{N}} (v^T (p - \bar{p}))^2 \\ &= \frac{1}{|\mathcal{N}|} \sum_{p \in \mathcal{N}} v^T (p - \bar{p}) (p - \bar{p})^T v \\ &= v^T \left(\frac{1}{|\mathcal{N}|} \sum_{p \in \mathcal{N}} (p - \bar{p}) (p - \bar{p})^T \right) v\end{aligned}$$

Minimizing Covariance

$$\begin{aligned}\arg \min_v \sigma^2(v) &= \frac{1}{|\mathcal{N}|} \sum_{p \in \mathcal{N}} (v^T (p - \bar{p}))^2 \\ &= \frac{1}{|\mathcal{N}|} \sum_{p \in \mathcal{N}} v^T (p - \bar{p}) (p - \bar{p})^T v \\ &= v^T \left(\frac{1}{|\mathcal{N}|} \sum_{p \in \mathcal{N}} (p - \bar{p}) (p - \bar{p})^T \right) v \\ &= v^T C_V\end{aligned}$$

Smooth Normals

For normals n_i, n_j of adjacent patches i, j : $n_i^T n_j \approx 1$.

Smooth Normals

For normals n_i, n_j of adjacent patches i, j : $n_i^T n_j \approx 1$.

Since approximating this is NP-hard, we achieve an approximate solution with an MST:

Part 2: The Laplacian Operator

Definition

Def.: Laplacian Operator

For a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$, the **Laplacian Operator** Δ is defined as

$$\Delta f := \operatorname{div}(\nabla f) = \nabla \cdot (\nabla f)$$

where $\nabla f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is the *gradient of f* and $\operatorname{div}(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}$ is the *divergence* of a vector field.

Definition

Def.: Laplacian Operator

For a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$, the **Laplacian Operator** Δ is defined as

$$\Delta f := \operatorname{div}(\nabla f) = \nabla \cdot (\nabla f)$$

where $\nabla f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is the *gradient of f* and $\operatorname{div}(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}$ is the *divergence* of a vector field.

In cartesian coordinates, the Laplacian is the *trace* of the Hessian matrix:

$$\Delta f = \operatorname{tr} \left(\left(\frac{\partial^2 f}{\partial x_j \partial x_i} \right)_{1 \leq i, j \leq n} \right) = \sum_{i=1}^d \frac{\partial^2 f}{\partial x_i^2}.$$

Definition

Def.: Laplacian Operator

For a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$, the **Laplacian Operator** Δ is defined as

$$\Delta f := \operatorname{div}(\nabla f) = \nabla \cdot (\nabla f)$$

where $\nabla f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is the *gradient* of f and $\operatorname{div}(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}$ is the *divergence* of a vector field.

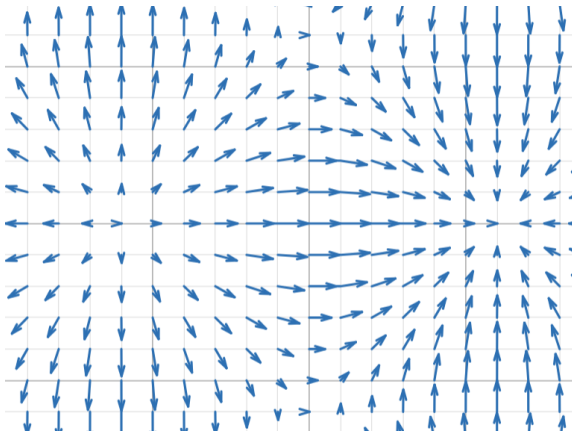
In cartesian coordinates, the Laplacian is the *trace* of the Hessian matrix:

$$\Delta f = \operatorname{tr} \left(\left(\frac{\partial^2 f}{\partial x_j \partial x_i} \right)_{1 \leq i, j \leq n} \right) = \sum_{i=1}^d \frac{\partial^2 f}{\partial x_i^2}.$$

For $d = 1$, it is the second derivative: $\Delta f = f''$.

Intuition

Divergence: A measure of "how much vectors spread out"



Derivation

In the discrete case, we can derive the Laplacian by using finite differences:

$$\begin{aligned}\Delta f(x) &= \frac{\partial^2}{\partial x^2} f(x) \\ &\approx \frac{\partial}{\partial x} \left(\frac{1}{2} f(x+1) - f(x-1) \right) \\ &= \frac{1}{2} \left(\frac{\partial}{\partial x} f(x+1) - \frac{\partial}{\partial x} f(x-1) \right) \\ &\approx \frac{1}{2} ([f(x+1) - f(x)] - [f(x) - f(x-1)]) \\ &= \frac{1}{2} (f(x+1) - 2f(x) + f(x-1)).\end{aligned}$$

Generalization to graphs

For a graph $G = (V, E)$ we can define the neighborhood for every $v_i \in V$ as

$$\mathcal{N}(v_i) = \{v_j \mid (v_i, v_j) \in E\}.$$

Generalization to graphs

For a graph $G = (V, E)$ we can define the neighborhood for every $v_i \in V$ as

$$\mathcal{N}(v_i) = \{v_j \mid (v_i, v_j) \in E\}.$$

With a value f_i associated with every v_i , the Graph Laplacian is

$$L(v_i) = \sum_{v_j \in \mathcal{N}(v_i)} (f_j - f_i) = \sum_{v_j \in \mathcal{N}(v_i)} (f_j) - \deg(v_i) f_i$$

or in matrix form

$$L = A - D$$

where $A_{i,j} = \begin{cases} 1 & \text{if } (i,j) \in E \\ 0 & \text{otherwise} \end{cases}$ is the adjacency matrix and

$D_{i,j} = \delta_{i,j} \cdot \deg(v_i)$ is the degree matrix.

Laplace Smoothing

Since the Laplacian gives us the direction toward the neighborhood average, we can use it to smooth a polygon or mesh:

$$\begin{pmatrix} v_1^{(t+1)} \\ \vdots \\ v_n^{(t+1)} \end{pmatrix} = (I + \lambda L) \begin{pmatrix} v_1^{(t)} \\ \vdots \\ v_n^{(t)} \end{pmatrix}, \quad \lambda > 0$$

Laplace Smoothing

Since the Laplacian gives us the direction toward the neighborhood average, we can use it to smooth a polygon or mesh:

$$\begin{pmatrix} v_1^{(t+1)} \\ \vdots \\ v_n^{(t+1)} \end{pmatrix} = (I + \lambda L) \begin{pmatrix} v_1^{(t)} \\ \vdots \\ v_n^{(t)} \end{pmatrix}, \quad \lambda > 0$$

Problem: this contracts the mesh toward the global average (a single point).

Taubin Smoothing

Solution: after contracting, expand the mesh again.

$$\begin{pmatrix} v_1^{(t+1)} \\ \vdots \\ v_n^{(t+1)} \end{pmatrix} = (I + \mu L)(I + \lambda L) \begin{pmatrix} v_1^{(t)} \\ \vdots \\ v_n^{(t)} \end{pmatrix}, \quad \mu < 0, \quad \lambda > 0, \quad |\mu| > |\lambda|$$

Laplace-Beltrami Operator

Problem with using the Graph Laplacian on surface meshes: only considers vertex connectivity, not the actual mesh geometry.

Laplace-Beltrami Operator

Problem with using the Graph Laplacian on surface meshes: only considers vertex connectivity, not the actual mesh geometry.

If we interpret the mesh surface as a *manifold* $\mathcal{M}$ and the function $f : \mathcal{M} \rightarrow \mathbb{R}$ as the *temperature* at every point in $\mathcal{M}$, then $\Delta_{\mathcal{M}} f$ should describe how the temperature changes as it *diffuses* over $\mathcal{M}$.

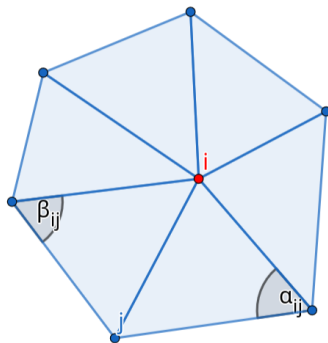
This is what the **Laplace-Beltrami Operator** does.

Derivation of the Laplace-Beltrami Operator

$$\Delta_{\mathcal{M}} v_i := \lim_{\text{diam}(A) \rightarrow 0} -\frac{\nabla A(v_i)}{A(v_i)} = -2\kappa_{\mathcal{M}} n_i$$

Laplace-Beltrami Operator

$$L(v_i) = \frac{1}{2A_i} \sum_{j \in \mathcal{N}(i)} (\cot \alpha_{ij} + \cot \beta_{ij})(v_j - v_i)$$



Laplace-Beltrami Operator

$$L(v_i) = \frac{1}{2A_i} \sum_{j \in \mathcal{N}(i)} (\cot \alpha_{ij} + \cot \beta_{ij})(v_j - v_i) \quad \Rightarrow L = M^{-1}S$$

